

Full length article

Patronus: Statistical decision-making for detecting volumetric application-layer DoS attacks

Hussain M.J. Almohri

Department of Computer Science, Kuwait University, P.O. Box 5969, Kuwait City 13060, Kuwait

ARTICLE INFO

Dataset link: <https://github.com/kussl/patronus>

Keywords:

Adaptive decision making
Denial-of-service detection
Anomaly detection
Parameter optimization

ABSTRACT

In a flooding Denial of Service (DoS) attack against the application layer, the adversary reduces the availability of the victim service through large volumes of requests that are individually structured like normal user traffic but collectively exhaust system resources. Prior work commonly frames such attacks as anomalies in network data and addresses them with machine learning models, behavioral analytics, statistical analysis, or adaptive thresholds. Many of these approaches depend on labeled attack data or attack-specific tuning, which limits their ability to generalize as traffic and attack patterns evolve. This article develops a statistical decision framework that requires no labeled attack data or a machine learning model. It relies only on a small benign traffic sample and a single belief value, supplied by the system administrator, that expresses the expected level of attack evidence in that sample. From this input, an optimization step fits a set of traffic signals that respond to elevation in structural request patterns rather than to attack-specific signatures. The signals are used in an online, evidence-based decision model that accumulates and decays evidence over time to classify windows of traffic as benign or under attack. The design of Patronus, a prototype of the framework, is presented and intended to complement existing intrusion detection systems. Evaluations on a real-world DoS dataset show that Patronus detects volumetric attack campaigns with high true positive rates and a low false positive rate, and that its leaky evidence model achieves stable attack detection.

1. Introduction

Application-layer distributed denial of service (DDoS) attacks exploit a web server's vulnerability to sudden large influxes of requests or gradual load-building distributed request campaigns. Although a classical attack pattern, the problem continues to harm modern web server software stacks. Mitigating these attacks starts with detection through an intrusion detection system (IDS), followed by a defensive action such as limiting request rates or blocking Internet Protocol (IP) addresses. Anomaly detection is the state of the art and has been studied as the primary method for detecting DDoS attacks (Sharif, 2023; Almaraz-Rivera et al., 2022). The methodology depends on building models of benign traffic, often requiring explicit labeling, against which incoming traffic is compared.

Although learning models achieve high classification accuracy, they remain limited by parameter weights tuned to the network conditions observed during training (Yang et al., 2024; Anley et al., 2024). An IDS based on a deep model can undergo scheduled retraining as new data become available. However, retraining is time-consuming, requires careful data isolation under varying threat models, and depends on system administrator intervention. Further, models can overfit to

particular request patterns, producing stale decision boundaries that reduce the model's ability to adapt to new traffic. A second classical problem facing both machine learning and statistical analysis of network traffic is parameter tuning. Arbitrary parameterization risks misrepresenting attack traffic and missing attacks, yet systematic tuning must balance this risk against the burden it places on the system administrator.

We present Patronus, a decision support system that complements conventional machine learning-based anomaly detection by providing model-agnostic, training-free threat monitoring built on a set of structural application-layer traffic signals. Patronus observes raw application-layer data such as Hypertext Transfer Protocol (HTTP) requests to assess the state of the application, treating every incoming request as a potential attack-campaign indicator. This assumption sidesteps the classification problem and lets Patronus work as an orthogonal decision-support layer for existing classification models. Rather than deciding whether a single request is benign or malicious, a judgment that depends entirely on the training process, Patronus treats requests as probabilistic contributors to an accumulating body of attack evidence. As evidence grows during an attack campaign, Patronus

E-mail address: almohri@cs.ku.edu.kw.

<https://doi.org/10.1016/j.cose.2026.105161>

Received 18 June 2026; Received in revised form 3 August 2026; Accepted 8 September 2026

Available online 14 September 2026

0167-4048/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

generates alerts based on the system administrator's initial belief about normal traffic, aggregate statistics over the traffic signals, and the evidence accumulation and decay parameters. The decision model decays this evidence automatically once the attack campaign stops, allowing the system to re-arm without manual intervention.

At its core, Patronus uses a modified Ornstein–Uhlenbeck (OU) process (Usher and McClelland, 2001), a gradual, cumulative confidence-building model in which temporal signals such as inter-arrival time and structural signals such as request path are extracted from raw HTTP access logs. As evidence compounds over time, attack confidence accumulates accordingly. To address waves of attacks, the OU process incorporates a leak parameter that controls how quickly past evidence decays, allowing the system to forget, retain, or amplify alertness over configurable time scales. This decay is the source of Patronus's adaptive behavior. The detector raises confidence as an attack persists and lowers it as traffic returns to normal, without retraining or human input. A further flexible aspect of Patronus is that all parameters of the confidence-building algorithm, including the leak, are derived from a single belief value provided by the system administrator, expressing an abstract estimate of the system's current attack exposure. This is not to say Patronus requires no setup.

Previous works have explored adaptive intrusion detection through human-in-the-loop feedback (Wang et al., 2004; Yu et al., 2008), confidence forwarding (Ippoliti et al., 2016), and Markov decision processes (Hu et al., 2020). Recent approaches employ deep neural networks for concept drift adaptation (Shyaa et al., 2024) and incremental attack discovery (Lu et al., 2025). These works have demonstrated the need for dynamic intrusion detection and presented various strategies to achieve it. However, they share a common reliance on labeled data, traffic profiles, or expert-tuned parameters to bridge raw anomaly signals and intrusion decisions. Patronus addresses this gap by transforming continuous anomaly indicators into actionable attack evidence through a single belief value.

A smaller body of work pursues training-free anomaly detection more directly. REPLICAWATCHER (Khairi et al., 2024) detects anomalous containers without labeled training data by measuring feature dissimilarity. Patronus shares its independence from labeled data but differs in both application domain and method, applying continuous attack-evidence accumulation to web request traffic rather than dissimilarity scoring on container behavior. Patronus also differs from unsupervised deep learning detectors such as D-PACK (Hwang et al., 2020), which require training a neural model on traffic to classify attacks. Patronus builds no machine learning or deep learning model at all and instead detects attacks from the deviation of live traffic from a small benign sample, fitting its parameters by solving a single optimization problem.

This work presents an in-depth study of methods for building cumulative attack evidence, argues for a modified OU process referred to as the leaky drift–diffusion model (LDDM), and introduces a novel and feasible optimization problem for efficient model parameterization. The optimization problem results in an initial set of hyperparameters that are used by the system at runtime. Specifically, the contributions of this work are as follows.

1. The design of Patronus, including its attack detection lifecycle, a model of heuristic-based attack signal generation from raw HTTP data, and a formal attack model addressed by Patronus (Section 3).
2. A set of traffic-agnostic structural signals that respond to elevation in request patterns rather than to attack-specific signatures (Section 5).
3. A decision support model based on the leaky drift–diffusion (LDDM) process, with a comparison against memoryless and accumulator decision models showing that LDDM attains similar detection while remaining stable without the manual resets the accumulator models require (Section 4).

4. The classification results and sensitivity analysis of the model on the CIC-IDS-2017 dataset (Sharafaldin et al., 2018) (Section 6).

Patronus is evaluated on the widely used CIC-IDS-2017 dataset with 483,008 extracted set of HTTP request traces spanning three days of benign and DoS attack traffic. The evaluation tests the sensitivity and robustness of the decision model and compares three alternative detection engines. The simulations show that Patronus detects volumetric attack campaigns with a true positive rate of 98% to 100% and a false positive rate of 1% to 5%.

2. Related work

Intrusion detection under concept drift has been an ongoing problem due to the nature of dynamic network data and the ever-changing attack landscape (Shyaa et al., 2024). Much of the recent work on intrusion and denial-of-service detection focuses on building advanced and fine-tuned deep learning models (Halbouni et al., 2022), which enrich the anomaly detection component. That said, adaptive decision support models are necessary because intrusion detection systems often degrade in real network environments (Yang et al., 2024).

This section presents various approaches to detection, decision-making, and mitigation for intrusion detection and ends with a list of gaps in the state of the art addressed by Patronus. Table 1 gives an overview of the related work. These works are categorized based on the core mechanism for approaching the intrusion detection problem, namely training-free detection, adaptive detection, decision-making under uncertainty, and network-layer reconfiguration.

Training-Free Detection. Earlier work such as CPM (Wang et al., 2004) and ADAT (Yu et al., 2008) share with our work the objective of training-free intrusion detection. CPM (and similarly the works by (Olufowobi et al., 2019; Leu and Yang, 2005)) uses the nonparametric cumulative sum model (CUSUM (Page, 1954)) to continuously decide whether an attack campaign is going on. Another work that has shown the possibility of using adaptive models such as CUSUM is WAVE-CUSUM in which wavelet analysis as a pre-processing filter is used in conjunction with CUSUM to perform network-level intrusion detection (Callegari et al., 2012). CIDS also uses CUSUM to detect abnormal shifts in in-vehicle messages for fingerprinting Electronic Control Units (Cho and Shin, 2016).

Ippoliti et al. come closest to our work. The authors observed that an online feedback loop is key to high-accuracy detection of anomalies without relying on training and labeled data (Ippoliti et al., 2016). They further include a confidence forwarding scheme based on the idea of *concept drift* and share our argument that not every anomaly is necessarily an attack on the system. Unlike the confidence forwarding approach, Patronus uses optimization-based parameter initialization from a single belief value, eliminating traffic profiling and heuristic tuning.

REPLICAWATCHER (Khairi et al., 2024) shares with Patronus the goal of detecting anomalies without labeled training data. It targets anomalous containers by measuring feature dissimilarity, whereas Patronus targets application-layer attack campaigns through continuous attack-evidence accumulation over web request data.

Adaptive Detection. ReCDA attempts to solve the problem by introducing self-supervised representation enhancement and weakly-supervised classifier tuning to improve the robustness of decision making in IDS (Yang et al., 2024). Unlike classical approaches (such as (Sharif, 2023; Almaraz-Rivera et al., 2022)) that focus on improving detection models, Nakip et al. propose a deep neural-network model that uses a self-supervised process to adapt to change (Nakip and Gelenbe, 2024). Their work is tested in Internet of Things (IoT) environments and uses deep learning. Patronus also aims for reducing human intervention, eliminating time-consuming learning phases, and enabling models that respond to change. That said, Patronus provides decision support modeling for application layer IDS. The concepts

Table 1

Comparison of adaptive intrusion-detection systems, grouped by the mechanism underlying their approach to detecting, deciding, and mitigating intrusions: training-free detection, adaptive detection, decision-making under uncertainty, and network-layer reconfiguration.

Work	Domain	Core concept
Training-Free detection		
CPM (Wang et al., 2004)	Network	CUSUM change detection
WAVE-CUSUM (Callegari et al., 2012)	Network	CUSUM change detection
CIDS (Cho and Shin, 2016)	Automotive	CUSUM change detection
Ippoliti et al. (Ippoliti et al., 2016)	Network	Confidence forwarding
REPLICAWATCHER (Khairi et al., 2024)	Containers	Feature dissimilarity
Adaptive detection		
ReCDA (Yang et al., 2024)	Network	Self-supervised representation enhancement
Nakup et al. (Nakup and Gelenbe, 2024)	IoT	Self-supervised deep neural network
Lu et al. (Lu et al., 2025)	Smart grid	Correlation-based incremental detection
ADAT (Yu et al., 2008)	Network	Fuzzy-logic rule adjustment
Barna et al. (Barna et al., 2014)	Network	Queueing-theory traffic prediction
D-PACK (Hwang et al., 2020)	Network	Unsupervised deep learning
Decision-Making under uncertainty		
Hu et al. (Hu et al., 2020)	Network	POMDP with Bayesian attack graphs
Cao et al. (Cao et al., 2024)	Network	Deep reinforcement learning
Kazeminajafabadi (Kazeminajafabadi and Imani, 2024)	Network	POMDP and optimization
Network-Layer reconfiguration		
Poseidon (Zhang et al., 2020)	Data plane	Switch reconfiguration
Jaqen (Liu et al., 2021)	Data plane	Programmable switch reconfiguration
NetHCF (Li et al., 2019)	Data plane	Hop-count filtering
ZAPDOS (Misa et al., 2024)	Data plane	Epoch-based prefix refinement
Patronus (this work)	Web	Optimization-based decision support

are complementary in principle, and their work can be embedded in Patronus as an anomaly detection system. The work of Lu et al. shares our concern but uses correlations to provide an incremental approach to intrusion detection aimed at discovering new attacks in smart grids (Lu et al., 2025).

ADAT integrates fuzzy-logic controllers and autonomously updates rule confidences in response to verified false alarms. It relies on human-in-the-loop expert feedback to adjust the fuzzy knowledge base, so it depends on the accuracy of the rule base and requires a significant user knowledge base to be effective.

The work of Barna et al. is another step toward effective and adaptive intrusion detection (Barna et al., 2014). Their work predicts the nature of the upcoming traffic to dynamically generate and refine traffic filtering rules based on mathematical queueing theory. Their algorithm aims at detecting the start of DoS attacks and generating mitigating rules, by flagging deviations of observed behavior from a queueing-theory model of expected behavior. Patronus expands the anomaly indicator approach and focuses on the quality of the detection decision rather than on producing mitigation rules.

Patronus also differs from unsupervised deep learning detectors such as D-PACK (Hwang et al., 2020), which depends on training a neural model on traffic to profile traffic patterns and flag anomalies. D-PACK also eliminates the need for traffic labeling, however, it requires deep learning model training. As shown in Section 6, our evaluations confirm that the accumulation-based decision engine used in Patronus outperforms conventional unsupervised learning methods.

Decision-Making Under Uncertainty. (Hu et al., 2020) and Kazeminajafabadi (Kazeminajafabadi and Imani, 2024) propose to use a partially observable Markov decision process (POMDP) for making defense decisions against unknown intrusion attacks. Specifically, the model by Hu et al. captures uncertainties in attack propagation via Bayesian attack graphs (BAGs), where nodes represent vulnerable machines and edges encode probabilistic exploit paths. They handle unknown exploit likelihoods and impacts by maintaining belief states, estimating transition probabilities with Thompson sampling, and choosing defense actions with reinforcement learning. (Cao et al., 2024) introduce a deep reinforcement learning approach and use a Markov decision process to select moving-target-defense actions against

unknown attacks in software defined networks. Patronus avoid attack propagation as it only requires attack evidence collected from historical data windows while deciding on the current window of traffic.

Network-Layer Reconfiguration. Poseidon (and similarly Jaqen (Liu et al., 2021), NetHCF (Li et al., 2019), and ZAPDOS (Misa et al., 2024)) manages change in the network layer by employing reconfiguration when new data arrives (Zhang et al., 2020). The system focuses on providing programming support for configuring programmable switches to defend against attacks. NetHCF (Li et al., 2019) and the work by Wagner et al. (Wagner et al., 2021) share the approach Patronus uses to produce anomaly indicators. NetHCF flags packets whose observed hop count deviates from the learned IP-to-hop-count mapping. In contrast, Patronus focuses on providing decision support at the application layer. Although solving a distinct problem, Patronus uses an iterative window-based approach to adapt to changes, similar to ZAPDOS (Misa et al., 2024).

Gaps Addressed by Patronus. Prior approaches share a common dependence on labeled data, traffic profiles, or expert-tuned parameters to support decision making in intrusion detection systems, or they decide memorylessly on the current view of the traffic. Several gaps therefore remain.

1. The state of the art lacks a decision-focused model that, instead of training full benign and attack profiles, accumulates evidence as an attack emerges, maintains the attack state for as long as the campaign continues, and decays the evidence once the campaign subsides. Existing accumulators such as CUSUM build evidence but provide no intrinsic decay, and they diverge without manually imposed resets. This dynamic modeling of the incoming traffic enables sound decisions for as long as the benign traffic profile remains largely unchanged. Further, adaptive models such as CUSUM, although studied for anomaly detection in general, have not been studied in the context of application-layer intrusion detection systems.
2. Existing DoS detection methods require training, labels, or expert annotations. Even though many achieve high detection accuracy, they face bottlenecks in their data needs and in parameter initialization.

3. Beyond initialization, expert involvement adds a recurring cost. Expert input has value in directing a method toward the right detections, but it becomes a hurdle when the underlying intrusion detection system must adapt, since every adaptation renews the demand for expert effort.
4. The adaptive and reconfiguration-based defenses surveyed above operate at the network layer, leaving application-layer request semantics outside their mechanisms. Attackers can specifically target the application layer and bypass network-layer defenses entirely.

3. Adaptive attack detection in patronus

This section provides the problem definition and the design overview of Patronus, followed by a formal model of application-layer denial of service attacks, showing the attacker's objectives in the form of an optimization problem.

3.1. Problem definition

Web services can be targeted at various layers with the objective of denying service to benign, legitimate users. A specific class of denial of service occurs at the application layer (Tripathi and Hubballi, 2021). In application-layer campaigns, the attacker produces large volumes of requests to exhaust the target server's resources, pushing it toward saturation.

Before deploying a mitigation solution, the server needs a reliable mechanism for determining whether the live incoming traffic stream is part of a volumetric denial-of-service campaign. The server must, therefore, make timely decisions about the state and intent of its clients. The server's core advantage is the availability of incoming HTTP-layer requests, which mainstream web server software such as Nginx and Apache collect efficiently.

Patronus addresses the problem of deciding whether the recently observed live traffic constitutes a denial-of-service attack campaign, using only HTTP request data. An effective solution requires a decision-making mechanism that prioritizes a low false positive rate and does not depend on manual or automated data labeling. Patronus assumes a powerful volumetric threat model in which the attacker can flood the server with application-layer requests, exploiting open firewall policies. The attacker has knowledge of the server's internal mechanisms and is aware of its DoS vulnerability points.

The solution approach is to develop a decision model that uses a series of attack signals, computed from the HTTP request traffic, leading to the accumulation or decay of an *attack evidence* value. Patronus maintains and updates this value at fixed window intervals. The model is evaluated by its ability to classify incoming traffic windows as benign or attack. The main metrics are the True Positive Rate (TPR) and the False Positive Rate (FPR) of the decision engine at the window level, along with the detection latency, measured as the number of windows between the onset of an attack and the first alert.

The two metrics, TPR and FPR, may have inverse effects. Increasing the belief value P raises the system's sensitivity, detecting more attack windows at the cost of more false alarms, while decreasing it has the opposite effect, as the sensitivity analysis in Section 6.11 shows. Patronus prioritizes a low and predictable false positive rate, since excessive false alarms overwhelm operators and erode trust in an intrusion detection system, and the belief value gives the administrator a single control for selecting the operating point along this trade-off.

3.2. Design overview

Patronus provides an adaptive and statistical decision support engine for intrusion detection systems that

1. works at the application layer with raw network access data (web server logs),
2. provides a decision support system based on the Ornstein–Uhlenbeck process model (Section 4.1),
3. produces a rule-based anomaly indicator score using a rich set of features (Section 4.2), and
4. systematically performs optimization-based parameterization using a belief value and a parameter bootstrapping process (Section 5).

Patronus operates in two stages. In the offline stage, the parameters of the system are tuned and during the online stage the system detects intrusions and produces alerts. In the following, the components of the Patronus architecture (Fig. 1) are described.

3.2.1. Parameter optimizer

During offline parameter initialization, a *Parameter Optimizer* determines two sets of parameters. The first is the parameters of the decision engine, which accumulates attack evidence and compares it against a threshold to decide whether the recently observed traffic shows evidence of an intrusion. The second is the set of signal weights, which combine the individual anomaly signals into the anomaly indicator Ω (formally defined in Section 4.2) that drives the evidence accumulation. The Parameter Optimizer uses a sample of benign HTTP traffic, collected during normal operation of the web service, to establish the benign baseline.

The Parameter Optimizer (Section 5) solves an optimization problem that computes the decision engine parameters and the signal weights so that benign traffic produces low attack evidence consistent with the administrator's belief value, while a diversity constraint prevents the weight from concentrating on any single signal. The optimization is performed offline, producing the set of parameters that are used during the online processing of incoming requests.

3.2.2. Data manager and signal extractor

The anomaly indicator model, the decision model, and the optimized parameter sets are used during the online anomaly detection stage of Patronus. When the online operation starts, Patronus uses its *Data Manager* to read, interpret, and process the stream of HTTP access traffic produced by the web server software. As a middleware, the Data Manager's output is consumed by a key online component, the *Signal Extractor*, which consults the signal model to produce normalized signal values.

As shown in Fig. 2, each signal has a dedicated function that uses the raw request data from a single window to produce one signal value. For example, the maximum per-IP request rate is computed as the highest per-IP request count in the window divided by the window span. Each computed signal value is then passed through an exponential normalizer that maps it into $[0, 1]$. The normalization is monotonically increasing and saturates toward 1 for large signal values. As a result, stronger deviations from the benign baseline yield higher normalized values. The normalized signal values are passed to the Decision Engine, which combines them using the optimized weights to produce the anomaly indicator and the accumulated attack evidence.

3.2.3. The decision engine

Patronus uses the Signal Extractor's output as the input to the Decision Engine E (formally defined in Section 4.1). In the first stage, the Decision Engine aggregates (Fig. 3) the normalized signal values using the optimized signal weights. This signal aggregator produces the anomaly indicator.

The anomaly indicator is similar in spirit to the histogram-based outlier score (Goldstein and Dengel, 2012), in that each signal contributes an independent anomaly score that is then aggregated into a single value. The anomaly indicator $\Omega \in [0, 1]$ is continuous, and a value near $\Omega = 1$ indicates that the observed requests in the current

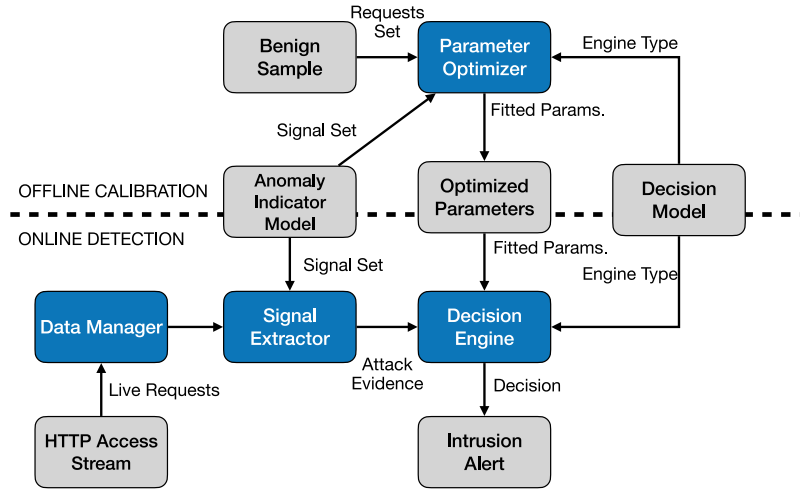


Fig. 1. The architecture of Patronus. The system uses a decision engine and an anomaly indicator model. In the offline parameter initialization stage, a benign dataset is used to select decision model parameters and the weights of anomaly signals. In the online stage, the system uses the incoming stream of HTTP traffic to compute the anomaly indicator value Ω in an evidence accumulation process, producing intrusion alerts when Ω exceeds a threshold.

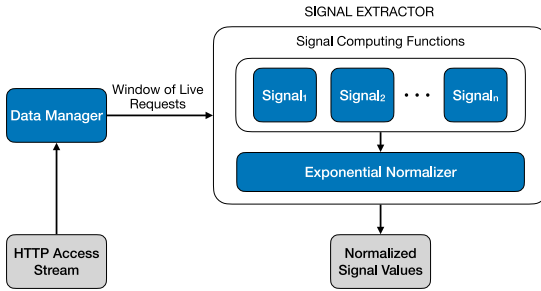


Fig. 2. The internal mechanism of the Signal Extractor where each signal is collected, processed, and normalized to be consumed by the Decision Engine.

time window deviate strongly from the benign baseline across the measured signals, while Ω near 0 indicates traffic consistent with that baseline.

The evidence calculator then uses the Decision Model's mechanism to produce the accumulated attack evidence value, following one of the models discussed in Section 4.1. Using this evidence value, the decision producer applies the optimized threshold from the Parameter Optimizer to produce the final intrusion alert.

3.3. A model of denial of service attacks

A denial of service (DoS) attack on a network service aims at reducing the availability of service when accessed by benign users. More formally, let $c(s, t)$ be a function that determines the capacity of the service s at any point in time t . The capacity can be a complex relationship of various system resources or a simple indicator such as network bandwidth. The objective of an adversary can be formalized as follows.

$$J(t_0, t_1) = \int_{t_0}^{t_1} c(s, t) dt - \lambda(t_1 - t_0), \quad t_0 < t_1 \quad (1)$$

$$\min_{t_0 < t_1} J(t_0, t_1). \quad (2)$$

The integral captures the capacity over a time interval, and $\lambda \geq 0$ controls how much the attacker values the duration of the attack, where $\lambda = 0$ places the sole emphasis on attack strength. The objective is subject to the adversary's own resources and the defense capabilities of the target service s . The constraint on the attacker's capabilities is

an upper bound on the total cost of N attack actions $u_{a,k}$, given budget B_a ,

$$\sum_{k=1}^N u_{a,k} \leq B_a. \quad (3)$$

The defense capabilities depend on two decisions. First, the defense must *decide* whether the service is under attack, ideally predicting the persistence of the attack. Second, it must decide whether action is necessary and choose the correct response from the available toolbox. The attacker's objective is therefore also constrained by the defense measures available to the victim s ,

$$c(s, t) \leq c_{\text{base}} + \eta(E; X) \beta, \quad (4)$$

where $\eta(E; X) \in [0, 1]$ is the accuracy of the decision engine E on input data X , $\beta \in [0, 1]$ is the remaining fraction of the defense budget, and c_{base} is the baseline capacity under attack. As the inequality encodes, defense capability depends primarily on the accuracy η of the decision engine, which determines how reliably the state of attack on s is identified over the interval. This accuracy is determined by the input data. The objective of the defense is thus to maximize the decision engine's accuracy over the interval,

$$\max_{E, t} \eta(E; X). \quad (5)$$

The accuracy of E depends on a reliable supply of information about the state of the network. A decision engine may perform well at one moment yet report an unreliable value when the characteristics of the network data change. Patronus addresses this by accumulating evidence over time and decaying it as traffic returns to normal, maintaining a decision support value that reflects the current state of the traffic.

4. The decision support model

This section develops the formal foundations for computing the attack evidence (Section 4.1) and the anomaly indicators (Section 4.2), the core functions of Patronus. Section 5 then discusses the parameterization of each model. The model computes attack evidence from application-layer request signals, which represent both the known benign traffic and the live, unclassified incoming traffic. The decision support model treats deviation from the known benign signals as evidence of an attack.

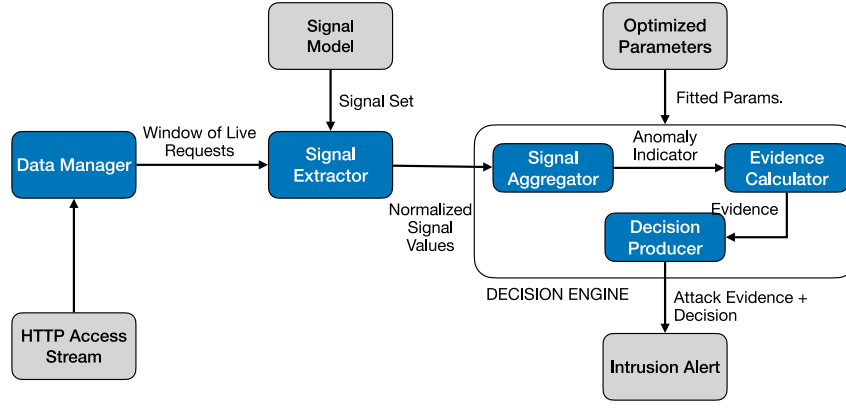


Fig. 3. The internal mechanism of the Decision Engine where the intrusion alert decision is produced given the signal values for one window of live stream data.

4.1. Computing evidence of attack

Formally, consider $X_t \in \mathcal{X}^{n_t}$, where n_t is the number of requests in window t , as the request data for window t . Let

$$\mu_t = (f_{t,1}, \dots, f_{t,m}) \in [0, 1]^m \quad (6)$$

be the vector of m summary features (indicators) extracted from X_t , where each $f_{t,i}$ is normalized to $[0, 1]$. The anomaly indicator is a linear score

$$\Omega_t = \sum_{i=1}^m \theta_i f_{t,i}, \quad (7)$$

with coefficients $\theta_i \geq 0$ normalized so that $\Omega_t \in [0, 1]$ (enforced by $\sum_{i=1}^m \theta_i = 1$).

Given the anomaly indicator $\Omega_t \in [0, 1]$ at each time step t , Patronus uses a decision engine E to accumulate evidence and map it to a decision value d_t ,

$$E : (\Omega_t, x_{t-1}; \rho, v, \tau) \longrightarrow (x_t, d_t), \quad (8)$$

where x_{t-1} is the previous evidence state, ρ is the leak rate, v is the sensitivity to Ω_t , τ is the decision threshold, and x_t is the updated evidence. For a binary decision, $d_t \in \{0, 1\}$ with $d_t = \mathbf{1}\{x_t > \tau\}$.

As a simplified example, consider the mean request rate and the inter-arrival time as two features characterizing the requests in an interval. Their values are extracted from the data, normalized to $[0, 1]$, and passed to E , which maps the incoming values to a decision on whether the window exhibits malicious patterns.

The internal mechanism of E is a critical design choice that strongly affects the outcome. In this work we consider and evaluate (Section 6) several choices for E : a simple threshold decision, the Cumulative Sum (CUSUM) model (Page, 1954), and the Ornstein–Uhlenbeck process (Usher and McClelland, 2001; d’Afflisio et al., 2018), referred to in this paper as the leaky drift–diffusion model (LDDM).

A simple threshold reacts immediately to any anomaly but keeps no memory of past traffic. It compares the current window’s signal against the benign baseline and decides in isolation. CUSUM and LDDM instead retain awareness of past values. CUSUM accumulates evidence but has no intrinsic decay, so as the simulated accumulation in Fig. 4 shows, each attack wave drives CUSUM monotonically higher and it does not return to baseline once the attack subsides. LDDM, by contrast, grows or shrinks its evidence as attack waves come and go, which suits the detection of gradual and recurring attacks. The results of Section 6.9 show that this yields substantially lower false positive rates than the memoryless threshold under bursty traffic and pulse-wave attacks.

For the threshold engine, a single rule signals when the current score exceeds τ ,

$$d_t = \mathbf{1}\{\Omega_t \geq \tau\}. \quad (9)$$

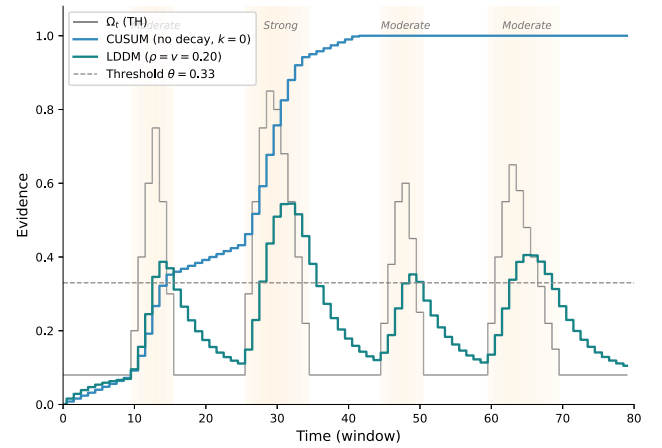


Fig. 4. Simulated accumulation of evidence in CUSUM and LDDM under attack waves.

CUSUM (Page, 1954; Moustakides, 1986) is a lightweight recursive detector with well-known optimality properties. Its evidence is built as

$$S_t = \max\{0, S_{t-1} + (\Omega_t - k)\}, \quad d_t = \mathbf{1}\{S_t > h\}, \quad (10)$$

relying on the previous value S_{t-1} and updating according to new anomaly information from Ω_t . The parameter k regulates the tolerable distance from the benign baseline.

Finally, the leaky drift–diffusion model (LDDM) accumulates evidence with a drift coefficient v that controls reliance on the instantaneous signal and a leak coefficient ρ that controls how long past evidence is retained. We adopt the deterministic form of the model, dropping the stochastic diffusion term, so the evidence x_t at time t is

$$x_t = (1 - \rho)x_{t-1} + v\Omega_t. \quad (11)$$

4.2. Computing the anomaly indicator

The anomaly indicator transforms raw HTTP request data into a continuous score that quantifies deviation from normal behavior. The guiding idea is to use signals that characterize request *patterns* with no reliance on request content, so the indicator responds to changes in traffic structure rather than to specific payloads or signatures. Observing the request logs over a time window (such as 30 s), Patronus computes a feature vector $f_{t,1}, \dots, f_{t,m}$. Each raw feature is mapped to $[0, 1]$ by the exponential normalization $g(x) = 1 - e^{-\alpha x}$, where α controls how quickly the score saturates toward 1. The aggregate anomaly indicator

Table 2

Anomaly indicator features. N is the number of requests in the window, w the window size, Δt the mean inter-arrival time, $\bar{b}$ the mean content length, $H(\cdot)$ the Shannon entropy, $H_{\max}$ the maximum entropy for the observed support, c_i the count for IP i , and σ/μ the coefficient of variation of inter-arrival times. Each raw value is normalized to $[0, 1]$ via $g(x) = 1 - e^{-ax}$, except header completeness, which is already a proportion.

Feature	Semantics	Raw Metric
Request rate	Density of requests in the time window	N/w
Inter-arrival time	Mean time gap between consecutive requests	$\overline{\Delta t}$
Payload size	Mean content length	$\bar{b}$
Connection density ratio	Ratio of unique source IPs to total requests	$ \text{IPs} /N$
Request entropy	Normalized entropy of request distribution across IPs	$H/H_{\max}$
HTTP method entropy	Normalized entropy of HTTP method distribution	$H/H_{\max}$
Path diversity	Normalized entropy of Uniform Resource Identifier (URI) path distribution	$H/H_{\max}$
Max IP request rate	Peak per-IP request rate in the window	$\max_i c_i/w$
Temporal burstiness	Coefficient of variation of inter-arrival times	σ/μ
Header completeness	Proportion of missing standard HTTP headers	$1 - N_{\text{present}}/(N \cdot 8)$

is the weighted sum

$$\Omega_i = \sum_{i=1}^m \theta_i f_{t,i}, \quad \sum_{i=1}^m \theta_i = 1, \quad \theta_i \geq 0, \quad (12)$$

where θ_i is the relative importance of feature i and each $f_{t,i}$ is the normalized value $g(\cdot)$ of its raw metric. The weights $\{\theta_i\}$ are determined by the bootstrap optimization of Section 5, which fits them on known benign traffic subject to the administrator's belief value.

The features are listed in Table 2. They capture complementary dimensions of HTTP traffic, namely volume (request rate, max IP request rate), timing (inter-arrival time, temporal burstiness), source distribution (connection density ratio, request entropy), resource targeting (path diversity), request structure (HTTP method entropy), payload characteristics (payload size), and client fingerprinting (header completeness). Larger α makes the score more sensitive to small deviations from the benign baseline. Features where low raw values indicate attacks, such as small payloads from scripted attacks, use the inverted form e^{-ax} so that a low raw value produces a high anomaly score.

Note that the feature set can be adapted to the application area or tuned to specific attack profiles, and it can be systematically filtered through feature selection (Raeisi et al., 2025). Feature selection is beyond the scope of this work.

5. Computing model parameters

Computing the parameters of the model is a central contribution of this article. We assume the system administrator has access to benign, representative traffic from the protected system. Together with a single belief value about the level of attack evidence in that traffic, the administrator fits the signal weights and the decision model parameters by solving a well-posed and efficiently solvable optimization problem.

Parameter bootstrapping adjusts the model parameters so that the evidence computed on a benign dataset matches the administrator's belief value. The process begins with a dataset X_0 of structured access logs containing benign traffic. The administrator provides a belief value $P \in [0, 1]$ that estimates the attack evidence they expect in X_0 . When X_0 is collected in a normal state, a small P expresses the expectation of little attack evidence.

Given X_0 and P , bootstrapping fits the anomaly indicator weights and the decision model parameters (for LDDM, the leak ρ and the drift v) by solving an optimization problem whose solution (i) brings the

mean benign evidence to the belief value P and (ii) spreads weight across the features rather than concentrating it on a few. The process first computes the mean of each feature over the benign dataset,

$$\bar{f}_i = \frac{1}{n} \sum_{i=1}^n f_{t,i}, \quad i = 1, \dots, m, \quad (13)$$

which enters the optimization as the coefficients on the weight variables. After solving, the evidence is evaluated on X_0 to set an initial alert threshold for use by an intrusion detection system. The procedure is summarized in Algorithm 1.

Algorithm 1 Optimization-based bootstrapping

Require: Bootstrap dataset X_0 , belief P , window size w
 Compute feature means $\bar{f}_i = \frac{1}{|X_0|} \sum_t f_{t,i}$ for $i = 1, \dots, m$
 Solve Problem (14) with $\{\bar{f}_i\}$ and P to obtain (θ_i, ρ, v)
 Evaluate evidence on X_0 : $x_t \leftarrow (1 - \rho)x_{t-1} + v\Omega_t$, where $\Omega_t = \sum_i \theta_i f_{t,i}$
 Set threshold $\tau \leftarrow \text{percentile}(\{x_t\}, 95)$
return $(\theta_i, \rho, v, \tau)$

Recall that the accumulated evidence follows the recursion $x_t = (1 - \rho)x_{t-1} + v\Omega_t$, where at each step a fraction ρ of the previous evidence leaks away and a fraction v of the current anomaly indicator Ω_t is added. When the input holds steady at the benign mean $\bar{\Omega} = \sum_i \theta_i \bar{f}_i$, the evidence settles at an equilibrium value where the amount leaking out each step equals the amount coming in. Setting $x_t = x_{t-1}$ in the recursion gives this equilibrium as $x_{\infty} = (v/\rho)\bar{\Omega}$. The optimizer fits the weights and the parameters ρ and v so that this equilibrium evidence for benign traffic equals the administrator's belief value P . The problem is

$$\begin{aligned} \max_{\theta, \rho, v} \quad & \frac{v}{\rho} \sum_{i=1}^m \theta_i \bar{f}_i \\ \text{s.t.} \quad & \frac{v}{\rho} \sum_{i=1}^m \theta_i \bar{f}_i \leq P, \\ & \sum_{i=1}^m \theta_i = 1, \\ & 0 \leq \theta_i \leq w_{\max} \\ & (i = 1, \dots, m), \quad \epsilon \leq \rho \leq 1, \quad 0 \leq v \leq 1, \end{aligned} \quad (14)$$

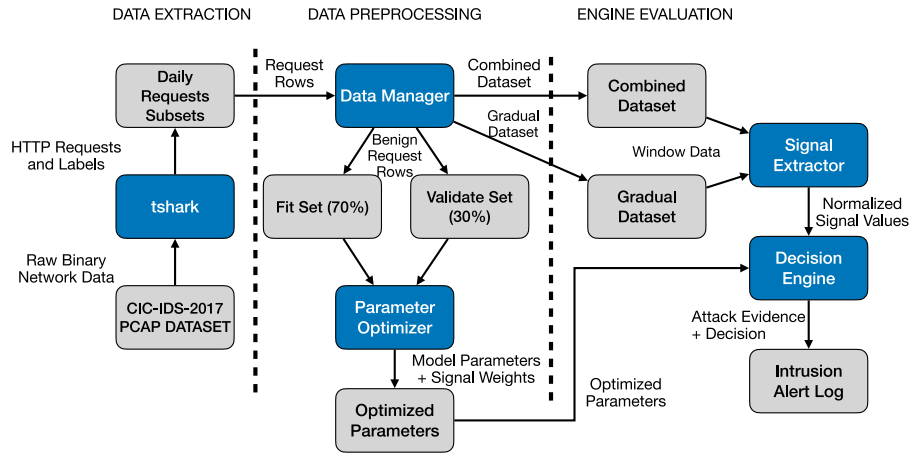


Fig. 5. The performance evaluation pipeline in which the raw data is extracted from the dataset and processed, and the behavior of the online Patronus intrusion detection is then simulated.

where $\bar{f}_i = \frac{1}{n} \sum_{t=1}^n f_{t,i}$ is the benign mean of feature i . The objective and the belief constraint share the same expression, so the objective drives the benign steady-state evidence up to the ceiling P , making the constraint bind and placing the operating point exactly at the administrator's chosen sensitivity.

In this formulation, the Benign traffic is close to the belief value, while attack traffic, which produces stronger signal deviations, accumulates evidence above it. The per-weight bound $\theta_i \leq w_{\max}$ diversifies the optimized weights. This is because no single weight may exceed $w_{\max}$. Thus, the optimizer distributes weight across multiple signals rather than concentrating it on any one, and as a result, detection does not depend on a single signal that an adversary could evade.

We solve Problem (14) with Sequential Least Squares Programming (SLSQP) (Kraft, 1988), an established method for constrained nonlinear programs (Lawson and Hanson, 1995; Joshy and Hwang, 2024). The feasible region is nonempty and compact: all decision variables lie in closed bounded intervals, the weight simplex $\sum_i \theta_i = 1$ with $\theta_i \geq 0$ is convex, and a feasible point is obtained by any admissible ρ and v with uniform weights $\theta_i = 1/m$, which satisfy $\Omega_i \in [0, 1]$ and the belief constraint for sufficiently small evidence. Since the feasible region is compact and the objective is continuous, a global optimum exists.

6. Evaluation

This section presents the results for evaluating Patronus. First, the section gives an overview of the CIC-IDS-2017 dataset (Section 6.1), followed by the data extraction (Section 6.2), preprocessing (Section 6.3), and engine evaluation (Section 6.4) stages of the evaluation pipeline (Fig. 5). After describing the experimental setup (Section 6.5), the classification results comparing the three decision engines, LDDM, CUSUM, and the memoryless threshold (Section 6.6) are presented. Then Patronus is evaluated under different benign calibration samples (Section 6.7), and the contribution of each signal group to detection performance is analyzed (Section 6.8). The Decision Engine is evaluated under a pulse-wave variant of the dataset (Section 6.9) and against unsupervised machine learning baselines (Section 6.10). Finally, the sensitivity of LDDM to its hyperparameters (Section 6.11) is reported along with a report on the computational overhead (Section 6.12).

6.1. Dataset

The evaluation is based on the real-world intrusion detection dataset CIC-IDS-2017, which is well established and widely studied in prior work (Ferrag et al., 2020; Zhang et al., 2019; Ferrag and Maglaras, 2020). We use three days of traffic containing both benign requests

and denial-of-service (DoS) attack requests. Unlike datasets that publish only derived flow records, CIC-IDS-2017 includes the raw packet captures alongside flow-level labels, which makes it possible to reconstruct the application-layer HTTP requests that Patronus operates on while retaining the published ground truth for evaluation. The dataset contains both single-source (Hulk, GoldenEye) and multi-source (DDoS) volumetric attacks against web applications, which are the attacks Patronus specifically addresses.

The stages and the pipeline of the evaluations are summarized in Fig. 5 and described next.

6.2. Data extraction

The dataset consists of binary packet captures (PCAP) recorded with Wireshark over daily periods, each spanning several hours and containing a mix of benign and attack traffic. Patronus operates on raw HTTP request data, classifying each incoming application-layer request as attack or benign and generating alerts that an intrusion detection system can consume. The dataset's native packet-level format requires preprocessing before it is usable by Patronus. Data extraction uses Wireshark's tshark command-line utility to extract HTTP requests from the packet captures, discarding lower-level traffic such as transport-layer ACK messages that Patronus does not use. The extracted requests are written in text form to a comma-separated values (CSV) file.

Table 3 reports the number of extracted attack and benign requests and the corresponding count of 30-second windows per category. Table 4 gives a per-day breakdown of the requests, and Table 5 reports the distribution of each attack type. Attacks are temporally concentrated but voluminous, occupying relatively few windows yet contributing the majority of requests, consistent with the bursty nature of volumetric attacks. Benign traffic, by contrast, is spread across many more windows at a lower request rate. Because the three capture days are not contiguous, a large number of windows contain no requests and the empty windows are counted as benign. Patronus targets volumetric attacks and the slow-rate attacks (slowhttptest, slowloris) and network-layer reconnaissance (portscan) fall outside its scope, as their signatures do not manifest as an elevation in the application-layer request signals.

6.3. Data preprocessing

After extraction, the HTTP request rows are loaded from the CSV file into a Python DataFrame for lightweight preprocessing. Timestamps are converted to Unix epoch seconds, and features irrelevant to the study are dropped, keeping `src_ip`, `content_length`, `uri`, `method`, `raw_first_line`, `label`, and `attack_type` in the loaded DataFrame.

Table 3

Dataset overview. The capture spans three non-contiguous days, so a large fraction of the 30-second windows contain no requests; these empty windows are counted as benign.

Property	Value	%
Total requests	483,008	100.0
BENIGN	220,385	45.6
DoS attack	262,623	54.4
Capture span	Jul 03–Jul 07 (3 days)	–
Total windows (30s)	12,491	100.0
BENIGN	12,341	98.8
MIXED	59	0.5
ATTACK	91	0.7

Table 4

Per-day request breakdown.

Day	Requests	% of total	% DoS
Mon	112,229	23.2	0.0
BENIGN	112,229		
Wed	220,172	45.6	75.8
BENIGN	53,341		
DoS	166,831		
Fri	150,607	31.2	63.6
BENIGN	54,815		
DoS	95,792		
Total	483,008	100.0	54.4

Table 5

Attack type distribution. Untyped DoS requests are labeled as attacks in the ground truth without a specific attack-type annotation.

Attack type	Count	% of DoS
Hulk	151,030	57.5
DDoS	94,614	36.0
GoldenEye	7917	3.0
SlowHTTPTest	337	0.1
PortScan	85	<0.1
Slowloris	4	<0.1
Total typed	253,987	96.7
Untyped DoS	8636	3.3

The prepared benign data is passed to the Parameter Optimizer, which uniformly samples 70% of the benign requests to fit the system parameters and holds out the remaining 30% to validate the fit. The optimizer solves the belief-constrained problem of Section 5 such that the mean benign evidence meets the administrator's belief value. The held-out 30% is used to confirm that the resulting parameters generalize to unseen benign traffic before they are accepted.

6.4. Engine evaluation

To evaluate the classification performance of the decision support models (Section 4), we implemented the Data Manager, the Signal Extractor, and the Decision Engine components of Patronus. The stream of HTTP traffic is simulated by having the Data Manager load successive windows of requests. In each iteration of the evaluation loop, the window data is passed to the Signal Extractor, which returns the normalized signal values. The signals used in the evaluation are specified in a configuration file, with each signal's weight precomputed during data preprocessing by the Parameter Optimizer, and the parameters remain fixed for the entire run.

The Decision Engine aggregates the normalized signal values into the anomaly indicator, accumulates the attack evidence according to the selected decision model, and decides whether the current window

is labeled as an intrusion. The outcome of each window's evaluation is written to a comma-separated values (CSV) file for subsequent performance analysis.

6.5. Experimental setup

All experiments use the CIC-IDS-2017 dataset. Depending on the evaluation task, we use the individual daily subsets (Monday, Wednesday, Friday), the combined dataset of the three days, or a synthetic dataset derived from the three days with gradual attack campaigns. Before each experiment, the system parameters are initialized by the Parameter Optimizer using the Wednesday subset, which contains both benign and attack traffic, unless otherwise specified. Preprocessing and benign sample selection follow the procedure described in Section 6.3.

Traffic is processed in non-overlapping 30-second windows, so the step size equals the window size. Except in the sensitivity analyses, the normalization parameter $\alpha = 0.08$ and the initial belief $P = 0.3$ are fixed during both parameter optimization and live evaluation. Once fitted, the parameters and signal weights remain fixed throughout the entire simulated network stream.

All experiments run on an Apple M2 chip (8-core, 4 performance and 4 efficiency cores) with 16 GB of unified memory, using Python 3.x with the NumPy and SciPy libraries for the optimization.

Patronus uses a core decision engine to produce intrusion alerts, and we evaluate several engines to situate its performance. The first is a memoryless threshold model, which produces an alert whenever the anomaly indicator exceeds a threshold determined during the optimization process. The second is CUSUM, a commonly used evidence-accumulation model without an intrinsic decay factor. The third is LDDM, the engine used by Patronus, which both accumulates and decays evidence over time. In addition, we compare these engines against unsupervised learning methods, providing a systematic evaluation of the design choices and their trade-offs.

All decision-engine parameters, including the threshold, CUSUM's slack and decision level, and LDDM's leak and drift, are fitted by the same optimization procedure (Algorithm 1) on the same benign sample. For the unsupervised learning baselines, the hyperparameters are chosen to match the initial belief value used by the decision engines. Fitting every baseline through a comparable procedure on identical data ensures that performance differences reflect the decision mechanism rather than tuning.

6.6. Classification results

To assess the classification performance of Patronus, we designed a simulation loop that mimics the runtime environment: the CIC-IDS-2017 requests are presented along their original timeline to a simulated request monitor, which feeds one of the studied decision engines (threshold, CUSUM, or LDDM) to classify each window as attack or benign. Each decision model is fitted using the optimization-based bootstrapping method on a benign subset of the Wednesday requests. Table 7 reports the fitted parameters and signal weights for each model.

Two observations stand out from Table 6. First, all three engines achieve high true positive rates (≥ 0.97) on the volumetric attacks, confirming that the signal model enables detecting these attacks effectively regardless of the decision engine. Second, the engines differ in how they remain stable over time. Without a manual per-window reset, CUSUM cannot decay its accumulated evidence and stays elevated through benign periods and produces a false positive rate of 0.84. A manual hard reset restores CUSUM to a usable operating point, but the reset is an external, schedule-imposed intervention. LDDM achieves comparable detection without any manual reset, because its leak parameter ρ provides bounded, principled forgetting intrinsic to the model. LDDM thus matches the detection of the reset-based engines while obtaining the lowest false positive rate, and it does so without the ad hoc reset the accumulator models depend on.

Table 6

Classification results across decision models (belief = 0.30) with hard reset where applicable.

Model	Reset	FPR	TPR
LDDM	intrinsic decay	0.0079	0.9780
TH	stateless	0.0343	1.0000
CUSUM	hard reset	0.0464	1.0000
CUSUM	none	0.8405	1.0000

Table 7

Fitted parameters and optimized signal weights for each decision model. All fits use belief=0.3 and $\alpha = 0.08$ on Wednesday benign data.

Parameter	TH	LDDM	CUSUM
ρ (leak)	–	0.4271	–
v (drift)	–	0.4120	–
k (slack)	–	–	0.1045
Threshold	0.6189	0.5033	0.6080
<i>Signal weights</i>			
request_rate	0.1331	0.1292	0.1012
inter_arrival_time	0.0672	0.0711	0.0988
payload_size	0.1396	0.1349	0.1015
conn_density	0.0605	0.0652	0.0985
request_entropy	0.0626	0.0670	0.0986
http_method_entropy	0.0625	0.0669	0.0986
path_diversity	0.0645	0.0687	0.0987
max_ip_rate	0.1347	0.1305	0.1013
temporal_burst	0.1414	0.1365	0.1015
header_compl	0.1340	0.1300	0.1012

6.7. Benign sample sensitivity

Detection engines can be susceptible to overfitting to a specific calibration sample. To measure LDDM's resilience to the choice of benign sample, we conducted an experiment in which the model is evaluated on the same combined dataset of Monday, Wednesday, and Friday, while the calibration each time uses the benign traffic of a different day.

Table 8 reports the results. The LDDM false positive rate ranges only from 0.0027 to 0.0128 across the three calibration sources, while the change in true positive rate and per-type detection remains negligible. The stateless threshold engine is similarly stable, with a false positive rate between 0.0091 and 0.0144 and unchanged detection. In contrast, CUSUM is highly sensitive to the calibration sample, with its false positive rate spanning 0.4942 to 0.6911.

These results confirm that LDDM is robust to the choice of benign calibration sample. Notably, calibration on Monday, which contains no attack traffic, yields the same detection as calibration on the attack-bearing days. Thus, the sample does not need to contain attacks nor originate from the same deployment period.

LDDM incurs up to a single window of detection latency, whereas the threshold and CUSUM engines detect immediately. This reflects LDDM's evidence accumulation, which requires elevation to persist across a window before evidence crosses the decision threshold. The one window delay is the cost of the temporal filtering that suppresses transient benign spikes, and it is minor given the 30 s window and the false positive reductions reported.

6.8. Signal subset sensitivity

The attack signals used in Patronus collectively drive the evidence accumulation process. They fall into four groups, measuring anomalies in the volume, timing, distribution, and content of the requests. To analyze the contribution of each group, we performed an ablation study using the LDDM engine in which the model is re-fitted on the Wednesday benign sample with one group removed from the signal set

each time, and evaluated on the combined dataset. Table 9 reports the results against the baseline that uses all four groups.

Two outcomes stand out. First, detection is largely insensitive to which group is removed, with the true positive rate ranging only from 0.9670 to 0.9780, indicating that no single group is essential for detecting volumetric attacks. Three of the four reduced models surprisingly detect marginally more attack windows than the full model, recovering two additional GoldenEye windows. Second, and in contrast, removing any group raises the false positive rate substantially, from 0.0027 with all signals to between 0.0084 and 0.0246. Thus, the signals contribute mainly to precision, suppressing false alarms rather than enabling detection, and the full model trades a small amount of detection for a threefold to ninefold reduction in false positives.

The cost of removal is not uniform across groups, and the ordering is informative. Removing the content signals mostly increases FPR to 0.0246, followed by distribution increasing FPR to 0.0168 and timing to 0.0113, while removing the volume signals costs the least (with FPR at 0.0084). The content and distribution signals contribute little to detecting the attacks but do most of the work in separating genuine campaigns from benign surges. TPR and FPR are largely dependent on different signal groups, motivating the use of all four rather than reducing the model to the signals that appear most attack-relevant.

The fitted parameters move in opposite directions across the ablations. The leak ρ falls to 0.3333 when distribution signals are removed but rises to 0.4717 when content signals are removed, so the loss of precision reflects the reduced information available to the model rather than a uniform shift in its parameters.

6.9. Classification under pulse-wave attacks

The CIC-IDS-2017 dataset presents attacks as abrupt, sustained bursts, under which the memoryless threshold and LDDM achieve similar classification performance (Section 6.6). These step-onset attacks are a property of how the dataset was generated and do not exercise the central capability of an evidence-accumulating model, which is to integrate signal over time rather than react to each window in isolation. Real volumetric campaigns are often not abrupt. Pulse-wave attacks rise, peak, and recede, and benign traffic itself is bursty, producing transient spikes that momentarily resemble an attack. A memoryless threshold cannot distinguish a brief benign spike from the onset of a sustained campaign, because it has no state beyond the current window. LDDM, by accumulating and decaying evidence, requires elevation to persist before it crosses threshold, which filters transient benign bursts that would trigger a threshold.

To evaluate this behavior, we constructed a dataset that re-sequences real CIC requests into a pulse-wave pattern. The benign traffic is the Monday capture and the attack traffic is a subset of the Wednesday Hulk requests, arranged into three Gaussian-shaped waves that gradually rise to a peak and decay. No requests are synthesized. The modified dataset only contains rearranged arrival timing, and the wave shape models documented pulse-wave DDoS behavior (Kovacs, 2017). Both the threshold and LDDM are fitted independently and optimally using the optimization formulation of this paper, each on a benign sample with its own α and signal weights, so the threshold is given its best operating point rather than a handicapped one. Table 10 reports the results.

On Segment A, which is pure benign traffic with no attacks, LDDM produces a false positive rate of 4.6% while the threshold produces 25.4%, a more than fivefold difference. The threshold false-alarms on roughly one in four benign windows because the bursty benign traffic repeatedly spikes above its cutoff, whereas LDDM's accumulated evidence does not cross threshold on transient spikes. The threshold's marginally higher detection rate (90.0% versus 85.0%) is a consequence of more aggressive detection, which also results in a far higher false positive rate. Compared at a matched false positive rate, the threshold's detection would fall below LDDM's. This result quantifies

Table 8

Sensitivity to the choice of benign calibration sample. Each engine is fitted independently on each day's benign traffic and evaluated on the same combined dataset (12,491 windows, 12,341 benign). All fits use belief $P = 0.30$. LDDM reports leak ρ and drift v ; CUSUM reports slack k ; TH is stateless.

Calibration	ρ	v	k	FPR	TPR	DDoS	GoldenEye	Hulk	Latency
<i>LDDM</i>									
Monday	0.4713	0.3719	–	0.0116	0.9780	39/40	16/17	34/34	0
Wednesday	0.4490	0.3929	–	0.0027	0.9670	39/40	14/17	34/34	1
Friday	0.4421	0.3990	–	0.0128	0.9780	39/40	16/17	34/34	0
<i>CUSUM</i>									
Monday	–	–	0.1652	0.4942	1.0000	40/40	17/17	34/34	0
Wednesday	–	–	0.1312	0.6604	1.0000	40/40	17/17	34/34	0
Friday	–	–	0.1244	0.6911	1.0000	40/40	17/17	34/34	0
<i>TH</i>									
Monday	–	–	–	0.0144	1.0000	40/40	15/17	34/34	0
Wednesday	–	–	–	0.0091	0.9890	40/40	15/17	34/34	0
Friday	–	–	–	0.0096	0.9890	40/40	15/17	34/34	0

Table 9

Signal ablation on LDDM. Each row removes one signal group, re-fits on the Wednesday benign sample, and evaluates on the combined dataset. $\bar{L}$ is mean detection latency in windows (30 s each). Groups: Volume = request_rate, max_ip_request_rate, Timing = inter_arrival_time, temporal_burstiness, Distribution = connection_density_ratio, request_entropy, http_method_entropy, path_diversity, Content = payload_size, header_completeness.

Removed group	ρ	v	FPR	TPR	DDoS	GoldenEye	Hulk	$\bar{L}$
Baseline (none)	0.4490	0.3929	0.0027	0.9670	39/40	14/17	34/34	1
Volume	0.4673	0.3751	0.0084	0.9670	38/40	14/17	34/34	1.0
Timing	0.4391	0.4011	0.0113	0.9780	39/40	16/17	34/34	0.5
Distribution	0.3333	0.4803	0.0168	0.9780	39/40	16/17	34/34	0.5
Content	0.4717	0.3709	0.0246	0.9670	39/40	16/17	34/34	0.5

Table 10

Fitted parameters and detection performance on the synthetic gradual-onset dataset. Dataset: 342k HTTP requests (67% attack), 1489 windows at 30s resolution, 12.4h continuous span. Segments: A (pure benign Monday traffic, 2 h), B (Monday background + 3 Gaussian-shaped Hulk attack waves of increasing peak density, 2 h), C (original Wednesday data time-shifted, 8.4h). Each model is fitted independently with its own α and signal weights. FPR is over all benign windows; Benign FPR is Segment A only.

Model	α	ρ	v	θ	FPR	TPR	Benign FPR
LDDM	0.0119	0.4588	0.3839	0.3800	0.0273	0.8500	0.0458
TH	0.1184	–	–	0.4780	0.1530	0.9000	0.2542

the superiority of evidence accumulation over a memoryless decision due to robustness to bursty benign traffic and gradual attack onsets, precisely the conditions that abrupt-onset benchmarks such as CIC-IDS-2017 do not contain.

6.10. Comparison with unsupervised machine learning baselines

The decision support engine used by Patronus can be compared with well-established unsupervised methods for anomaly detection. We compare LDDM, the main decision engine, against Isolation Forest, One-Class Support Vector Machine (SVM), and Local Outlier Factor (LOF).

The experiments use the combined dataset (Table 11) and the synthetic gradual-onset dataset (Table 12). All models are fitted on the same Wednesday benign sample using the same normalized signals. To match LDDM's belief value of 0.3, Isolation Forest uses 100 estimators with a contamination of 0.3, One-Class SVM uses a radial basis function kernel with $\nu = 0.3$, and LOF uses 20 neighbors with a contamination of 0.3.

On the combined dataset, LDDM achieves both the lowest false positive rate and the highest true positive rate, detecting attacks as accurately as the baselines while producing far fewer false positives (33 versus 185 to 398). On the gradual-onset dataset, LDDM dominates Isolation Forest and One-Class SVM on both axes, achieving a lower false positive rate and a higher true positive rate at once. LOF attains a marginally lower false positive rate than LDDM on this dataset, but only by failing to detect the majority of attacks, with its true positive rate

collapsing to 0.36. Across both datasets, no baseline matches LDDM's combination of high detection and low false positives.

6.11. Sensitivity to hyperparameters

We evaluate the model's sensitivity to two hyperparameters. The first is α , which controls the exponential normalization function $1 - \exp(-\alpha s)$ applied to a signal value s . Its magnitude affects the range over which signals are mapped and therefore the fitted parameters. We swept α over the range 0.02 to 0.5, fitting each model to the Wednesday benign subset with a 70/30 training and testing split, and selected the α for which the mean benign training score matches the administrator's belief target P . Selection therefore depends only on benign training data and never on the test false positive rate or attack detection. Table 13 reports the resulting operating points. Relative to the fixed $\alpha = 0.08$ used in Table 6, the tuned α shifts the operating point of LDDM toward a lower false positive rate with a small reduction in true positive rate, while the threshold and CUSUM engines remain largely unchanged. This reflects the expected trade-off between false alarms and detection as the normalization scale tightens.

Table 14 reports the effect of the second hyperparameter, the system administrator's initial belief value P , which expresses the expected level of attack evidence in the benign bootstrap set. Holding α fixed and refitting on the Wednesday benign data, we vary P and run the simulated monitor with the resulting parameters. Two effects are visible. First, P reshapes the LDDM parameters smoothly and predictably. As P increases, the leak ρ decreases and the drift v increases monotonically,

Table 11

Comparison of LDDM against unsupervised baselines fitted on benign data only, using the same normalized signals and the same calibration sample. $\bar{L}$ is mean detection latency in windows (30 s each). FP is the absolute number of false positives over the 12,341 benign windows. Decision thresholds are engine-specific and not comparable across rows, and are therefore omitted.

Model	ρ	v	FPR	FP	TPR	DDoS	GoldenEye	Hulk	$\bar{L}$
LDDM	0.4490	0.3929	0.0027	33	0.9670	39/40	14/17	34/34	1.0
Isolation Forest	—	—	0.0150	185	0.9560	40/40	11/17	34/34	0.8
One-Class SVM	—	—	0.0203	250	0.9560	40/40	11/17	34/34	0.2
LOF	—	—	0.0323	398	0.5055	8/40	1/17	34/34	5.3

Table 12

Unsupervised machine learning baselines on the synthetic gradual-onset dataset (342k requests, 1489 windows). Machine learning models are fitted on the same benign-only protocol as LDDM/TH, using the Wednesday benign sample. FP is the absolute number of false positives over the 1281 benign windows. $\bar{L}$ is mean detection latency in windows (30 s each); — indicates fewer than 3 episodes detected.

Model	ρ	v	FPR	FP	TPR	GoldenEye	Hulk	SlowHTTP	$\bar{L}$
LDDM	0.4588	0.3839	0.0273	35	0.8500	13/17	88/151	2/31	2.7
Isolation Forest	—	—	0.1647	211	0.8100	13/17	78/151	1/31	2.3
One-Class SVM	—	—	0.1366	175	0.8100	13/17	78/151	5/31	4.0
LOF	—	—	0.0258	33	0.3600	1/17	35/151	0/31	15.3

Table 13

Classification results after alpha hyperparameter fitting (belief=0.35), ranked by FPR.

Model	Tuned α	FPR	TPR
LDDM	0.0243	0.0005	0.9341
TH	0.0557	0.0343	1.0000
CUSUM	0.0557	0.8405	1.0000

which confirms that the belief value behaves as an interpretable sensitivity dial that trades evidence retention for responsiveness. Second, detection remains robust despite this parameter shift. LDDM sustains a high true positive rate across the range and declines only slightly at the largest P , while the false positive rate stays within a narrow band. This stability is partly attributable to the diversification constraint in the optimizer, which discourages skewed signal weightings.

6.12. Computational performance

The computational performance of Patronus is evaluated on the CIC-IDS-2017 dataset. Windows are set to 30 s intervals across all experiments, and LDDM is used as the detection engine. The reported times are for non-empty windows, since some windows contain no requests because the capture days are not contiguous. The choice of engine has a negligible effect on these measurements, as all engines produce a decision through similar constant-time operations.

Table 15 reports the time to process the three main stages of a single window, namely signal extraction, window lookup, and the decision engine, in milliseconds. As the table shows, the majority of processing time (90.5%) is spent on signal extraction, which includes reading the signals, normalizing their values, and updating the window data with the processed signals. Window lookup accounts for a further 8.8%, while the decision engine's constant-time computation requires less than 1% of the total. This confirms that the decision support layer that Patronus contributes adds negligible overhead to the signal processing that any application-layer detector must perform.

The resident memory growth per window is reported in Table 16. Patronus shows stable memory usage across the run, with 98.8% of windows exhibiting no growth at all and the total growth over the entire dataset remaining below 4 megabytes. This reflects the fact that the state Patronus carries between windows is constant in size and independent of the number of requests or the length of the stream, comprising only the signal weights, the scalar evidence value, and the model parameters. The detector therefore has a stable memory footprint irrespective of traffic volume or duration.

Finally, throughput is evaluated as the number of requests processed through the 30 s arrival windows. Table 17 reports the number of processed requests, the non-empty windows over which throughput was computed, and the total wall-clock time. Even in the worst case, the densest window in the dataset requires only 16.26 ms to process its 5317 requests, well under the 30 s window duration, so the implementation keeps pace with the incoming stream with a wide margin.

7. Conclusions and limitations

This work presented Patronus, a decision support system for application-layer intrusion detection. Patronus transforms raw HTTP access logs into traffic-agnostic signals that respond to elevation in request patterns rather than to attack-specific signatures, and accumulates them into attack evidence using a leaky drift-diffusion (Ornstein-Uhlenbeck) process. All parameters are initialized by solving a single efficient optimization problem from one administrator-supplied belief value, with no labeled training data.

The key contributions are (i) a set of elevation-based, traffic-agnostic signals fitted by a label-free optimization that requires only benign data and a single belief value, eliminating model training, (ii) a memory-aware decision model based on leaky drift-diffusion, with a comparison against memoryless and accumulator engines showing comparable detection while LDDM remains stable without the manual resets the accumulator models require, (iii) a demonstration that evidence accumulation reduces false positives under bursty traffic and gradual, pulse-wave attack onsets, where a memoryless threshold over-reacts to transient spikes, and (iv) a prototype implementation evaluated on the externally labeled CIC-IDS-2017 dataset across multiple days, published as an open source project.

Several directions remain open. Patronus targets volumetric attacks, and slow-connection attacks (such as slowloris and slowhttptest) and network-layer reconnaissance (such as port scanning) fall outside its scope because their behavior does not manifest as elevation in application-layer request signals. Extending Patronus to a multi-layer formulation, in which evidence from network and transport layers contributes to a global attack evidence value, would broaden coverage to these classes. Patronus does not yet include a response-policy module, and it can be coupled with deep learning anomaly detectors as an additional signal source to enrich its attack evidence computation.

Patronus also has limitations in addressing concept drift, low-rate attacks, and flash crowds. Patronus assumes that benign traffic is stationary, and it therefore requires parameter recalibration when the underlying benign traffic behavior drifts. Patronus can be extended to

Table 14

Belief (P) sweep results across LDDM, TH, and CUSUM engines with fitted model parameters, P95 threshold θ , and classification metrics. All fits use fixed $\alpha = 0.0800$ on Wednesday benign training data.

P	LDDM					TH			CUSUM (no hard reset)			
	ρ	v	θ	FPR	TPR	θ	FPR	TPR	k	θ	FPR	TPR
0.1	0.5616	0.2678	0.1803	0.0134	0.9890	0.5383	0.0103	0.9231	0.2378	0.3702	0.3257	1.0000
0.2	0.4991	0.3435	0.3117	0.0130	0.9890	0.4841	0.0092	0.9890	0.1845	0.4642	0.4447	1.0000
0.3	0.4490	0.3929	0.4383	0.0139	0.9780	0.5742	0.0091	0.9890	0.1312	0.5604	0.6604	1.0000
0.4	0.4070	0.4283	0.5677	0.0126	0.9780	0.6634	0.0097	0.9890	0.0779	0.6557	0.9998	1.0000
0.5	0.3717	0.4551	0.6952	0.0126	0.9780	0.7538	0.0091	0.9890	0.0246	0.7523	0.9998	1.0000

Table 15

Per-stage processing time over the 2617 non-empty windows of the combined CIC-IDS-2017 dataset (30 s windows, LDDM engine). Of the 12,491 windows spanning the capture period, 9874 contain no requests because the capture days are not contiguous, and these are excluded from the latency distribution.

Stage	Total (s)	Median (ms)	P95 (ms)	P99 (ms)	%
Signal extraction	1.9302	0.406	1.584	12.451	90.5
Window lookup	0.1872	0.031	0.034	0.044	8.8
Decision engine	0.0154	0.002	0.002	0.003	0.7
Total	2.1328	–	–	–	100.0

Table 16

Per-window resident memory growth over the full run. The decision state carried between windows is constant, comprising the weight vector, the scalar evidence value, and the model parameters.

Metric	Value
Win. with zero growth	12,336 of 12,491 (98.8%)
Win. with nonzero growth	155 (1.2%)
Median nonzero growth	16.0 KB
P95 nonzero growth	80.0 KB
Max. nonzero growth	224.0 KB
Total growth	3.88 MB

Table 17

Processing throughput. Latency and per-window throughput statistics are computed over the 2617 non-empty windows. Sustained throughput is total requests divided by total processing time over non-empty windows only.

Metric	Value
Requests processed	483,008
Non-empty windows	2617
Total wall-clock time	2.02 s
Median per-window latency	0.44 ms
P95 per-window latency	1.63 ms
P99 per-window latency	12.50 ms
Worst-case window latency	16.26 ms
Sustained throughput	239,635 req/s

include automatic sample refresh, given a method for sensing benign concept drift without depending on the detector's own classifications.

The system does not address low-rate attacks, such as algorithmic complexity attacks, under which the server does not observe large volumes of requests. This problem requires a different set of signal models that observe system resources rather than the volume of incoming requests.

Finally, because the signals are based on traffic elevation, Patronus is limited in distinguishing flash crowds from attacks. A sudden surge of legitimate traffic can resemble an attack campaign and trigger false positives. This problem can be addressed, for example, by time series analysis that predicts flash crowd windows and discounts their effect on the accumulated attack evidence, a mechanism the decision engine is flexible enough to incorporate.

CRediT authorship contribution statement

Hussain M.J. Almohri: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Project administration, Methodology, Funding acquisition, Formal analysis, Conceptualization.

Declaration of Generative AI Use

During the preparation of this work the author used Claude (Anthropic) in order to improve language, clarity, and style of the manuscript. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

Funding

This work was supported and funded by Kuwait University Research Grant No. [RS02/24].

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data analyzed in this study consists of publicly available CIC-IDS-2017 intrusion detection dataset. The full implementation of Patronus, including the decision support engine, simulator, and attack injection framework, is publicly available as an open-source project at <https://github.com/kussl/patronus>.

References

- Almaraz-Rivera, J.G., Perez-Diaz, J.A., Cantoral-Ceballos, J.A., 2022. Transport and application layer DDoS attacks detection to IoT devices by using machine learning and deep learning models. *Sensors* 22 (9), [http://dx.doi.org/10.3390/s22093367](https://doi.org/10.3390/s22093367).
- Anley, M.B., Genovese, A., Agostinello, D., Piuri, V., 2024. Robust DDoS attack detection with adaptive transfer learning. *Comput. Secur.* 144, 103962. [http://dx.doi.org/10.1016/j.cose.2024.103962](https://doi.org/10.1016/j.cose.2024.103962).
- Barna, C., Shtern, M., Smit, M., Tzerpos, V., Litoiu, M., 2014. Mitigating DoS attacks using performance model-driven adaptive algorithms. *ACM Trans. Auton. Adapt. Syst.* 9 (1), [http://dx.doi.org/10.1145/2567926](https://doi.org/10.1145/2567926).

- Callegari, C., Giordano, S., Pagano, M., Pepe, T., 2012. WAVE-CUSUM: Improving CUSUM performance in network anomaly detection by means of wavelet analysis. *Comput. Secur.* 31 (5), 727–735.
- Cao, Y., Liu, K., Lin, Y., Wang, L., Xia, Y., 2024. Deep-reinforcement-learning-based self-evolving moving target defense approach against unknown attacks. *IEEE Internet Things J.* 11 (20), 33027–33039.
- Cho, K.-T., Shin, K.G., 2016. Fingerprinting electronic control units for vehicle intrusion detection. In: 25th USENIX Security Symposium (USENIX Security 16). pp. 911–927.
- d’Afflisio, E., Braca, P., Millefiori, L.M., Willett, P., 2018. Detecting anomalous deviations from standard maritime routes using the ornstein-uhlenbeck process. *IEEE Trans. Signal Process.* 66 (24), 6474–6487.
- Ferrag, M.A., Maglaras, L., 2020. DeepCoin: A novel deep learning and blockchain-based energy exchange framework for smart grids. *IEEE Trans. Eng. Manage.* 67 (4), 1285–1297. <http://dx.doi.org/10.1109/TEM.2019.2922936>.
- Ferrag, M.A., Maglaras, L., Moschogiannis, S., Janicke, H., 2020. Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. *J. Inf. Secur. Appl.* 50, 102419. <http://dx.doi.org/10.1016/j.jisa.2019.102419>.
- Goldstein, M., Dengel, A., 2012. Histogram-based outlier score (HBOS): A fast unsupervised anomaly detection algorithm. *KI-2012: Poster Demo Track 1*, 59–63.
- Halbouni, A., Gunawan, T.S., Habaebi, M.H., Halbouni, M., Kartiwi, M., Ahmad, R., 2022. Machine learning and deep learning approaches for CyberSecurity: A review. *IEEE Access* 10, 19572–19585. <http://dx.doi.org/10.1109/ACCESS.2022.3151248>.
- Hu, Z., Zhu, M., Liu, P., 2020. Adaptive cyber defense against multi-stage attacks using learning-based POMDP. *ACM Trans. Priv. Secur.* 24 (1), <http://dx.doi.org/10.1145/3418897>.
- Hwang, R.-H., Peng, M.-C., Huang, C.-W., Lin, P.-C., Nguyen, V.-L., 2020. An unsupervised deep learning model for early network traffic anomaly detection. *IEEE Access* 8, 30387–30399.
- Ippoliti, D., Jiang, C., Ding, Z., Zhou, X., 2016. Online adaptive anomaly detection for augmented network flows. *ACM Trans. Auton. Adapt. Syst.* 11 (3), <http://dx.doi.org/10.1145/2934686>.
- Joshy, A.J., Hwang, J.T., 2024. PySLSQP: A transparent Python package for the SLSQP optimization algorithm modernized with utilities for visualization and post-processing. *J. Open Source Softw.* 9 (103), 7246. <http://dx.doi.org/10.21105/joss.07246>.
- Kazeminajafabadi, A., Imani, M., 2024. Optimal joint defense and monitoring for networks security under uncertainty: A POMDP-based approach. *IET Inf. Secur.* 2024 (1), 7966713.
- Khairi, A.E., Caselli, M., Peter, A., Continella, A., 2024. ReplicaWatcher: Training-less anomaly detection in containerized microservices. In: Proceedings of the ISOC Network and Distributed System Security Symposium. NDSS, <http://dx.doi.org/10.14722/ndss.2024.24286>.
- Kovacs, E., 2017. Pulse wave ddos attacks disrupt hybrid defenses. URL <https://www.securityweek.com/pulse-wave-ddos-attacks-disrupt-hybrid-defenses/>. (Accessed 20 November 2025).
- Kraft, D., 1988. A software package for sequential quadratic programming. *Deutsche Forschungs- und Versuchsanstalt für Luft- und Raumfahrt Koln: Forschungsbericht, Wiss. Berichtswesen d. DFVLR*, pp. 1–33.
- Lawson, C.L., Hanson, R.J., 1995. Solving Least Squares Problems. Society for Industrial and Applied Mathematics, <http://dx.doi.org/10.1137/1.9781611971217>.
- Leu, F.-Y., Yang, W.-J., 2005. Intrusion detection with CUSUM for TCP-based DDoS. In: *International Conference on Embedded and Ubiquitous Computing*. Springer, pp. 1255–1264.
- Li, G., Zhang, M., Liu, C., Kong, X., Chen, A., Gu, G., Duan, H., 2019. NETHCF: Enabling line-rate and adaptive spoofed IP traffic filtering. In: 2019 IEEE 27th International Conference on Network Protocols. ICNP, IEEE Computer Society, Los Alamitos, CA, USA, pp. 1–12. <http://dx.doi.org/10.1109/ICNP.2019.8888057>.
- Liu, Z., Namkung, H., Nikolaidis, G., Lee, J., Kim, C., Jin, X., Braverman, V., Yu, M., Sekar, V., 2021. Jaqen: A High-Performance Switch-Native approach for detecting and mitigating volumetric DDoS attacks with programmable switches. In: 30th USENIX Security Symposium (USENIX Security 21). USENIX Association, pp. 3829–3846. URL <https://www.usenix.org/conference/usenixsecurity21/presentation/liu-zaoxing>.
- Lu, Q., An, K., Li, J., Wang, J., 2025. Network intrusion detection for modern smart grids based on adaptive online incremental learning. *IEEE Trans. Smart Grid* 16 (3), 2541–2553. <http://dx.doi.org/10.1109/TSG.2025.3535949>.
- Misa, C., Durairajan, R., Gupta, A., Rejaie, R., Willinger, W., 2024. Leveraging prefix structure to detect volumetric DDoS attack signatures with programmable switches. In: 2024 IEEE Symposium on Security and Privacy. SP, pp. 4535–4553. <http://dx.doi.org/10.1109/SP54263.2024.00267>.
- Moustakides, G.V., 1986. Optimal stopping times for detecting changes in distributions. *Ann. Statist.* 14 (4), 1379–1387.
- Nakip, M., Gelenbe, E., 2024. Online self-supervised deep learning for intrusion detection systems. *IEEE Trans. Inf. Forensics Secur.* 19, 5668–5683. <http://dx.doi.org/10.1109/TIFS.2024.3402148>.
- Olufowobi, H., Ezeobi, U., Muhati, E., Robinson, G., Young, C., Zambreno, J., Bloom, G., 2019. Anomaly detection approach using adaptive cumulative sum algorithm for controller area network. In: Proceedings of the ACM Workshop on Automotive Cybersecurity. AutoSec ’19, Association for Computing Machinery, New York, NY, USA, pp. 25–30. <http://dx.doi.org/10.1145/3309171.3309178>.
- Page, E.S., 1954. Continuous inspection schemes. *Biometrika* 41 (1–2), 100–115. <http://dx.doi.org/10.1093/biomet/41.1-2.100>.
- Raeisi, Z., Maleki, H.R., Akbari, R., 2025. An entropy-based multi-objective feature selection method for network intrusion detection. *Clust. Comput.* 28 (12), 790.
- Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A., 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: *International Conference on Information Systems Security and Privacy*. pp. 108–116. <http://dx.doi.org/10.5220/0006639801080116>.
- Sharif, D.M., 2023. High-accurate application-layer DDoS attack detection using machine learning. In: 2023 International Conference on Engineering Applied and Nano Sciences. ICEANS, pp. 72–76. <http://dx.doi.org/10.1109/ICEANS58413.2023.10629670>.
- Shyaa, M.A., Ibrahim, N.F., Zainol, Z., Abdullah, R., Anbar, M., Alzubaidi, L., 2024. Evolving cybersecurity frontiers: A comprehensive survey on concept drift and feature dynamics aware machine and deep learning in intrusion detection systems. *Eng. Appl. Artif. Intell.* 137, 109143. <http://dx.doi.org/10.1016/j.engappai.2024.109143>.
- Tripathi, N., Hubballi, N., 2021. Application layer denial-of-service attacks and defense mechanisms: A survey. *ACM Comput. Surv.* 54 (4), 1–33.
- Usher, M., McClelland, J.L., 2001. The time course of perceptual choice: the leaky, competing accumulator model. *Psychol. Rev.* 108 (3), 550.
- Wagner, D., Kopp, D., Wichtlhuber, M., Dietzel, C., Hohfeld, O., Smaragdakis, G., Feldmann, A., 2021. United we stand: Collaborative detection and mitigation of amplification DDoS attacks at scale. In: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. pp. 970–987.
- Wang, H., Zhang, D., Shin, K.G., 2004. Change-point monitoring for the detection of DoS attacks. *IEEE Trans. Dependable Secur. Comput.* 1 (4), 193–208. <http://dx.doi.org/10.1109/TDSC.2004.34>.
- Yang, S., Zheng, X., Li, J., Xu, J., Wang, X., Ngai, E.C.H., 2024. ReCDA: Concept drift adaptation with representation enhancement for network intrusion detection. In: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. KDD ’24, Association for Computing Machinery, New York, NY, USA, pp. 3818–3828. <http://dx.doi.org/10.1145/3637528.3672007>.
- Yu, Z., Tsai, J.J.P., Weigert, T., 2008. An adaptive automatically tuning intrusion detection system. *ACM Trans. Auton. Adapt. Syst.* 3 (3), <http://dx.doi.org/10.1145/1380422.1380425>.
- Zhang, Y., Chen, X., Jin, L., Wang, X., Guo, D., 2019. Network intrusion detection: Based on deep hierarchical network and original flow data. *IEEE Access* 7, 37004–37016. <http://dx.doi.org/10.1109/ACCESS.2019.2905041>.
- Zhang, M., Li, G., Wang, S., Liu, C., Chen, A., Hu, H., Gu, G., Li, Q., Xu, M., Wu, J., 2020. Poseidon: Mitigating volumetric DDoS attacks with programmable switches. In: The 27th Network and Distributed System Security Symposium (NDSS 2020). <http://dx.doi.org/10.14722/ndss.2020.24007>.